

CHƯƠNG 3

THIẾT KẾ DỮ LIỆU VÀ TỐI ƯU TRUY VẤN

Thời lượng: 4 LT | 4 TH — Công nghệ: EF Core, PostgreSQL, Repository Pattern, Bogus

MỤC TIÊU CHƯƠNG

Sau khi hoàn thành chương này, sinh viên có khả năng:

Về kiến thức

- Phân biệt Active Record và Data Mapper — hai pattern ORM phổ biến; giải thích tại sao EF Core thuộc Data Mapper và ưu điểm của nó trong kiến trúc phân tầng.
- Hiểu và trình bày được Repository Pattern (mục đích, cấu trúc `IRepository<T>`, specific repository) và Unit of Work (giao dịch, `SaveChanges` một lần duy nhất).
- Thiết kế được mô hình dữ liệu quan hệ cho Culinary Blog: Recipe, Category, RecipeStep, RecipeIngredient với các quan hệ 1-N, N-N và Owned Entity.
- Sử dụng thành thạo Fluent API trong EF Core: `EntityTypeConfiguration<T>`, Value Conversion, Owned Entity, Cascade Delete, Concurrency Token.
- Nhận diện và khắc phục N+1 Query Problem; hiểu khi nào dùng Eager Loading, Explicit Loading và `AsNoTracking`.
- Hiểu cơ chế B-tree và GIN index; biết cách tạo và sử dụng PostgreSQL Full-Text Search với `tsvector/tsquery` trong EF Core.
- Sử dụng thư viện Bogus để sinh dữ liệu thực tế phục vụ phát triển và kiểm thử.

Về kỹ năng

- Implement đầy đủ `IRepository<T>`, `IUnitOfWork` và các implementation tương ứng, tích hợp vào hệ thống DI.
- Viết `EntityTypeConfiguration` cho tất cả entities: cấu hình index, relationship, owned entity, value conversion.
- Phân tích query performance bằng `EXPLAIN ANALYZE` trong PostgreSQL và áp dụng tối ưu hóa phù hợp.
- Xây dựng Full-Text Search endpoint cho recipe, xử lý tiếng Việt với `unaccent` extension.
- Tạo seeding data thực tế với Bogus cho toàn bộ Culinary Blog để phục vụ development.

Về thái độ

- Tuân thủ nguyên tắc "measure before optimize" — đo lường thực tế với `EXPLAIN ANALYZE` trước khi tối ưu.

- Có tư duy thiết kế schema "database-first quality, code-first workflow" — schema tốt từ đầu, tránh migration debt.

GIỚI THIỆU CHƯƠNG

Tầng dữ liệu là nền tảng của toàn bộ ứng dụng — mọi tính năng đều phải đọc hoặc ghi vào đây. Thiết kế tầng dữ liệu kém không chỉ ảnh hưởng đến hiệu năng mà còn kéo lùi toàn bộ codebase: schema khó thay đổi, query rối rắm, test phức tạp. Chương này tập trung vào hai mục tiêu song song: thiết kế đúng và truy vấn hiệu quả.

Chúng ta sẽ bắt đầu từ lý thuyết ORM và pattern tổ chức tầng dữ liệu (Repository + Unit of Work), sau đó thiết kế đầy đủ mô hình dữ liệu cho Culinary Blog với EF Core Fluent API. Phần trọng tâm là tối ưu truy vấn — từ N+1 Problem (cạm bẫy phổ biến nhất với ORM) đến các kỹ thuật profiling với EXPLAIN ANALYZE. Chương kết thúc với PostgreSQL Full-Text Search — một tính năng native mạnh mẽ thường bị bỏ qua khi developer chỉ dùng LIKE để tìm kiếm.

Liên kết với Chương 1–2: IApplicationDbContext được định nghĩa ở Chương 1 sẽ được hiện thực hóa đầy đủ ở đây. Repository Pattern bổ sung thêm một tầng abstraction giúp Unit Testing dễ hơn (Chương 5). Cấu trúc entity và index được thiết lập trong chương này là nền tảng cho Cache Strategy ở Chương 7.

Database

Trong chương này, chúng ta hoàn thiện toàn bộ tầng Infrastructure/Persistence của Culinary Blog. Sau khi hoàn thành, database schema sẽ ổn định và sẵn sàng cho việc phát triển tính năng từ các chương sau.

3.1. NGUYÊN LÝ ORM VÀ DATA MAPPER PATTERN

ORM (Object-Relational Mapping) là kỹ thuật ánh xạ giữa các đối tượng trong ngôn ngữ lập trình hướng đối tượng và các bảng trong cơ sở dữ liệu quan hệ. Thay vì viết SQL trực tiếp, developer thao tác với các class và object — ORM tự động sinh SQL tương ứng. Điều này tăng tốc phát triển nhưng cũng ẩn đi độ phức tạp của SQL nếu dùng không đúng cách.

3.1.1 Active Record Pattern vs Data Mapper Pattern

Có hai cách tiếp cận chính trong thiết kế ORM:

Tiêu chí	Active Record (Ruby on Rails)	Data Mapper (EF Core, NHibernate)
Cách tổ chức	Entity class chứa cả data lẫn database logic	Entity class chỉ chứa data; DbContext/Repository chứa database logic
Ví dụ truy vấn	Recipe.find(id) — class tự truy vấn DB	context.Recipes.Find(id) — DbContext làm trung gian
Coupling	Entity phụ thuộc vào DB framework	Entity độc lập, không phụ thuộc ORM
Khả năng test	Khó mock	Dễ mock qua interface (IApplicationDbContext)
Phù hợp với	CRUD đơn giản, rapid prototyping	Kiến trúc phân tầng, DDD, Clean Architecture
Trong giáo trình	Không dùng	☒ EF Core + Repository Pattern

Bảng 3.1. So sánh Active Record và Data Mapper Pattern

3.1.2 EF Core là một Data Mapper

Entity Framework Core triển khai Data Mapper thông qua DbContext: các Entity class hoàn toàn là POCO (Plain Old C# Object) — không có thuộc tính hay phương thức nào liên quan đến database. Mọi thao tác với database đều thông qua DbContext và DbSet<T>. Điều này giúp Domain Entities trong Chương 1 hoàn toàn độc lập với EF Core.

Khái niệm

Change Tracking là cơ chế cốt lõi của EF Core: khi bạn truy vấn entity từ DbContext, EF Core tự động theo dõi mọi thay đổi trên entity đó. Khi gọi SaveChangesAsync(), EF Core so sánh trạng thái hiện tại với snapshot ban đầu và sinh ra SQL UPDATE chỉ cho các field đã thay đổi. Đây là cơ chế mạnh mẽ nhưng cũng là lý do AsNoTracking() quan trọng khi chỉ đọc dữ liệu.

3.2. REPOSITORY PATTERN VÀ UNIT OF WORK

Trong Clean Architecture, Application Layer không nên phụ thuộc trực tiếp vào EF Core. Repository Pattern thêm một tầng abstraction giữa Application và Infrastructure: thay vì Handler dùng DbContext trực tiếp, Handler dùng IRepository<T> — một interface được định nghĩa trong Application Layer và implement trong Infrastructure Layer.

Unit of Work (UoW) đảm bảo tất cả các thay đổi trong một "đơn vị công việc" (một HTTP request, một use case) được lưu vào database trong một transaction duy nhất. Nếu một phần thất bại, toàn bộ sẽ được rollback — đảm bảo tính nhất quán dữ liệu.

3.2.1 Interface definitions — Application Layer

```
C# CulinaryBlog.Application/Contracts/Persistence/IRepository.cs
```

```
namespace CulinaryBlog.Application.Contracts.Persistence;

// Generic repository – thao tác CRUD cơ bản cho mọi entity
// T phải là class (không phải value type) để EF Core track được
public interface IRepository<T> where T : class
{
    // Tìm theo Primary Key – trả về null nếu không tìm thấy
    Task<T?> GetByIdAsync(Guid id, CancellationToken ct = default);

    // Lấy tất cả – dùng khi cần toàn bộ danh sách nhỏ (vd: Categories)
    Task<IReadOnlyList<T>> GetAllAsync(CancellationToken ct = default);

    // Trả về IQueryable để caller tự thêm Where, OrderBy, Include, ...
    // Linh hoạt cho các truy vấn phức tạp mà không cần hàng chục overloads
    IQueryable<T> Query();

    Task AddAsync(T entity, CancellationToken ct = default);
    void Update(T entity); // Không async vì chỉ đánh dấu Modified trong
    tracker
    void Remove(T entity); // Không async vì chỉ đánh dấu Deleted trong
    tracker
    Task<bool> ExistsAsync(Guid id, CancellationToken ct = default);
    Task<int> CountAsync(CancellationToken ct = default);
}
```

```
C# CulinaryBlog.Application/Contracts/Persistence/IRecipeRepository.cs
```

```
// Specific repository kế thừa IRepository<Recipe>
// Thêm các truy vấn domain-specific mà không thể diễn đạt gọn qua IQueryable
public interface IRecipeRepository : IRepository<Recipe>
{
    // Lấy chi tiết recipe kèm tất cả navigation properties cần thiết
    // (thay vì caller phải tự Include từng cái)
    Task<Recipe?> GetDetailAsync(Guid id, CancellationToken ct = default);

    // Full-text search – tích hợp PostgreSQL tsvector
    Task<IReadOnlyList<Recipe>> SearchAsync(
        string searchTerm,
        int page = 1, int pageSize = 10,
        CancellationToken ct = default);

    // Lấy danh sách recipe theo category (phổ biến trong Culinary Blog)
    Task<IReadOnlyList<Recipe>> GetByCategoryAsync(
        Guid categoryId, int page = 1, int pageSize = 10,
        CancellationToken ct = default);

    // Kiểm tra slug đã tồn tại chưa (validate trước khi tạo)
    Task<bool> SlugExistsAsync(string slug, CancellationToken ct = default);
}
```

```
C# CulinaryBlog.Application/Contracts/Persistence/IUnitOfWork.cs
```

```
namespace CulinaryBlog.Application.Contracts.Persistence;
```

```
// Unit of Work tập trung tất cả repositories và quản lý transaction
// Handler chỉ cần inject IUnitOfWork – không cần inject từng repository riêng lẻ
public interface IUnitOfWork : IDisposable
{
    IRecipeRepository Recipes { get; }
    IRepository<Category> Categories { get; }
    IRepository<Comment> Comments { get; }

    // Lưu tất cả thay đổi vào DB – chỉ gọi MỘT LẦN ở cuối use case
    Task<int> SaveChangesAsync(CancellationToken ct = default);

    // Transaction thủ công – dùng khi cần kiểm soát commit/rollback
    Task BeginTransactionAsync(CancellationToken ct = default);
    Task CommitTransactionAsync(CancellationToken ct = default);
    Task RollbackTransactionAsync(CancellationToken ct = default);
}
```

3.2.2 Implementation — Infrastructure Layer

```
C# CulinaryBlog.Infrastructure.Persistence.Repositories.Repository.cs

using CulinaryBlog.Application.Contracts.Persistence;
using Microsoft.EntityFrameworkCore;

namespace CulinaryBlog.Infrastructure.Persistence.Repositories;

// Base implementation – mọi specific repository kế thừa class này
// Giảm code duplication: không cần viết lại GetByIdAsync, CountAsync, ...
public class Repository<T> : IRepository<T> where T : class
{
    protected readonly ApplicationDbContext Db;
    protected readonly DbSet<T> DbSet;

    public Repository(ApplicationDbContext db)
    {
        Db = db;
        DbSet = db.Set<T>();
    }

    public async Task<T?> GetByIdAsync(Guid id, CancellationToken ct = default)
        => await DbSet.FindAsync([id], ct);

    public async Task<IReadOnlyList<T>> GetAllAsync(CancellationToken ct = default)
        => await DbSet.AsNoTracking().ToListAsync(ct);

    // IQueryable cho phép caller chain thêm điều kiện – lazy evaluation
    public IQueryable<T> Query() => DbSet.AsQueryable();

    public async Task AddAsync(T entity, CancellationToken ct = default)
        => await DbSet.AddAsync(entity, ct);

    // Update và Remove chỉ đánh dấu trạng thái trong Change Tracker
    // Thực sự ghi vào DB khi SaveChangesAsync() được gọi
    public void Update(T entity) => DbSet.Update(entity);
```

```

public void Remove(T entity) => DbSet.Remove(entity);

public async Task<bool> ExistsAsync(Guid id, CancellationToken ct = default)
    => await DbSet.FindAsync([id], ct) is not null;

public async Task<int> CountAsync(CancellationToken ct = default)
    => await DbSet.CountAsync(ct);
}

```

C# CulinaryBlog.Infrastructure.Persistence.UnitOfWork.cs

```

using CulinaryBlog.Application.Contracts.Persistence;
using CulinaryBlog.Infrastructure.Persistence.Repositories;
using Microsoft.EntityFrameworkCore.Storage;

namespace CulinaryBlog.Infrastructure.Persistence;

public class UnitOfWork : IUnitOfWork
{
    private readonly ApplicationDbContext _db;
    private IDbContextTransaction? _transaction;

    // Lazy initialization – repository chỉ tạo khi được truy cập lần đầu
    private IRecipeRepository? _recipes;
    private IRepository<Category>? _categories;
    private IRepository<Comment>? _comments;

    public UnitOfWork(ApplicationDbContext db) => _db = db;

    // Lazy properties – share cùng DbContext instance trong request
    public IRecipeRepository Recipes
        => _recipes ??= new RecipeRepository(_db);
    public IRepository<Category> Categories
        => _categories ??= new Repository<Category>(_db);
    public IRepository<Comment> Comments
        => _comments ??= new Repository<Comment>(_db);

    // Một SaveChangesAsync duy nhất → một database roundtrip duy nhất
    public Task<int> SaveChangesAsync(CancellationToken ct = default)
        => _db.SaveChangesAsync(ct);

    public async Task BeginTransactionAsync(CancellationToken ct = default)
        => _transaction = await _db.Database.BeginTransactionAsync(ct);

    public async Task CommitTransactionAsync(CancellationToken ct = default)
    {
        await _transaction!.CommitAsync(ct);
        _transaction = null;
    }

    public async Task RollbackTransactionAsync(CancellationToken ct = default)
    {
        await _transaction!.RollbackAsync(ct);
        _transaction = null;
    }
}

```

```

public async ValueTask DisposeAsync()
{
    if (_transaction is not null)
        await _transaction.DisposeAsync();
    await _db.DisposeAsync();
}
}

```

💡 Lưu ý

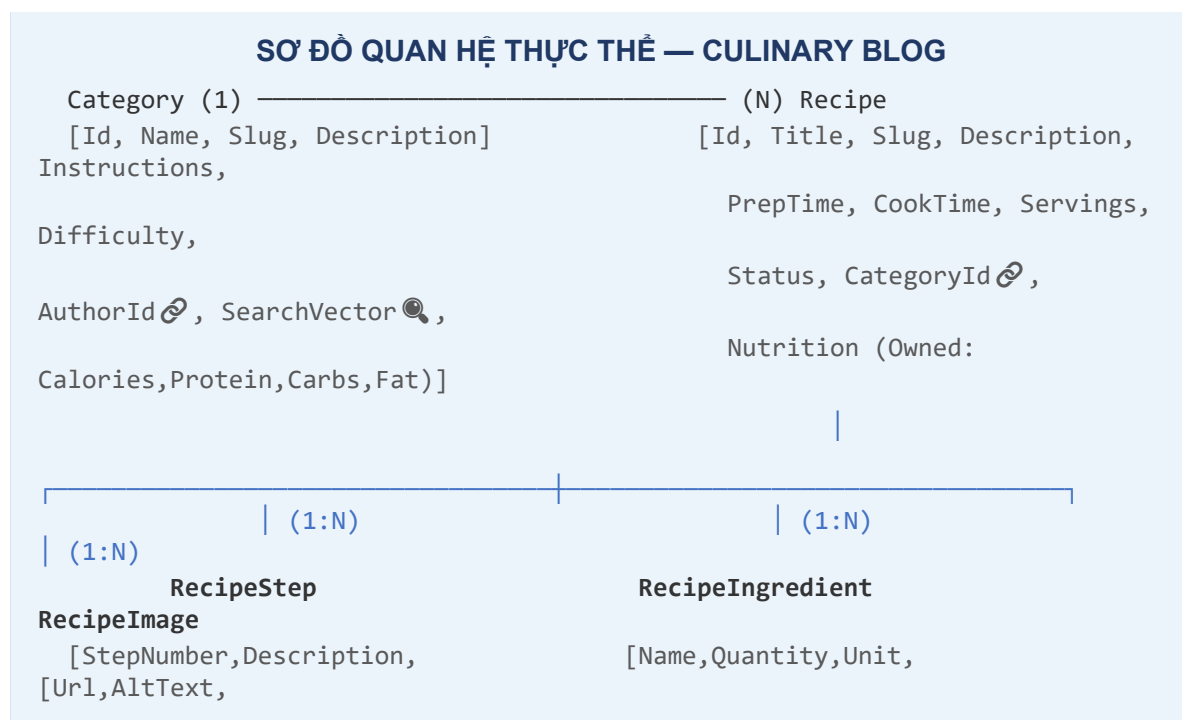
Với UnitOfWork, các CQRS Handler không inject IApplicationDbContext trực tiếp nữa (như Chương 1), mà inject IUnitOfWork. Trong unit test, dễ dàng mock IUnitOfWork với InMemoryDatabase hoặc NSubstitute. Đây là lý do tách Repository khỏi DbContext — testability. Chương 5 sẽ hướng dẫn chi tiết cách viết unit tests cho handlers.

3.3. THIẾT KẾ CƠ SỞ DỮ LIỆU POSTGRESQL

Culinary Blog cần lưu trữ công thức nấu ăn với cấu trúc phức tạp: mỗi recipe có nhiều bước thực hiện theo thứ tự, nhiều nguyên liệu với số lượng và đơn vị, nhiều ảnh minh họa, và thông tin dinh dưỡng tùy chọn. Ngoài ra còn cần hỗ trợ tìm kiếm toàn văn bản (Full-Text Search) và phân loại theo danh mục.

3.3.1 Sơ đồ quan hệ thực thể (ERD)

Bảng dưới đây thể hiện các entities chính và quan hệ giữa chúng. Ký hiệu: 🔑 = Primary Key, 🔗 = Foreign Key, 🔍 = Indexed/Search.



```

    DurationMinutes,ImageUrl,
    IsPrimary,RecipeId [
    RecipeId [
    Notes,RecipeId [

```

Hình 3.1. Sơ đồ quan hệ thực thể Culinary Blog (đơn giản hóa)

3.3.2 Domain Entities — Recipe và các thực thể liên quan

```

C# CulinaryBlog.Domain/Entities/Recipe.cs

namespace CulinaryBlog.Domain.Entities;

// Enum – Lưu vào DB dạng string (dễ đọc, không bị ảnh hưởng khi thêm giá trị)
public enum DifficultyLevel { Easy, Medium, Hard }
public enum RecipeStatus { Draft, Published, Archived }

public class Recipe
{
    public Guid Id { get; private set; }
    public string Title { get; private set; } = string.Empty;
    public string Slug { get; private set; } = string.Empty;
    public string? Description { get; private set; }
    public string? Instructions { get; private set; } // HTML hoặc
    Markdown
    public int PrepTimeMinutes { get; private set; }
    public int CookTimeMinutes { get; private set; }
    public int Servings { get; private set; }
    public DifficultyLevel Difficulty { get; private set; }
    public RecipeStatus Status { get; private set; } =
    RecipeStatus.Draft;

    // Owned Entity – Lưu trong cùng bảng Recipes (không JOIN)
    public RecipeNutrition? Nutrition { get; private set; }

    // Foreign Keys
    public Guid CategoryId { get; private set; }
    public string AuthorId { get; private set; } = string.Empty;

    // Navigation properties
    public Category? Category { get; private set; }
    public ApplicationUser? Author { get; private set; }
    public ICollection<RecipeStep> Steps { get; private set; } = [];
    public ICollection<RecipeIngredient> Ingredients { get; private set; } = [];
    public ICollection<RecipeImage> Images { get; private set; } = [];

    public DateTime CreatedAt { get; private set; }
    public DateTime? UpdatedAt { get; private set; }

    // Computed column trong PostgreSQL – EF Core readonly
    // EF Core không ghi vào đây; PostgreSQL tự cập nhật khi Title/Description
    thay đổi
    public NpgsqlTsVector? SearchVector { get; private set; }

    // Concurrency token – phát hiện khi hai user cùng sửa một recipe
    [Timestamp]
    public byte[]? RowVersion { get; private set; }

```

```

protected Recipe() { }

public static Recipe Create(string title, string? description,
    Guid categoryId, string authorId,
    int prepTime, int cookTime, int servings, DifficultyLevel difficulty)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(title, nameof(title));

    return new Recipe
    {
        Id = Guid.NewGuid(),
        Title = title.Trim(),
        Slug = SlugHelper.Generate(title),
        Description = description?.Trim(),
        CategoryId = categoryId,
        AuthorId = authorId,
        PrepTimeMinutes = prepTime,
        CookTimeMinutes = cookTime,
        Servings = servings,
        Difficulty = difficulty,
        Status = RecipeStatus.Draft,
        CreatedAt = DateTime.UtcNow,
    };
}

// Business method: publish recipe (có thể thêm validation rules ở đây)
public void Publish()
{
    if (Steps.Count == 0)
        throw new DomainException("Recipe phải có ít nhất 1 bước thực
hiện.");
    Status = RecipeStatus.Published;
    UpdatedAt = DateTime.UtcNow;
}

// Cập nhật thông tin dinh dưỡng (Owned Entity)
public void SetNutrition(decimal calories, decimal protein, decimal carbs,
decimal fat)
=> Nutrition = new RecipeNutrition(calories, protein, carbs, fat);
}

```

C# CulinaryBlog.Domain/Entities/RecipeStep.cs + RecipeIngredient.cs

```

// — RecipeStep

public class RecipeStep
{
    public Guid Id { get; private set; }
    public int StepNumber { get; private set; } // Thứ tự bước (1, 2, 3...)
    public string Description { get; private set; } = string.Empty;
    public int? DurationMinutes { get; private set; } // Thời gian ước tính cho bước này
    public string? ImageUrl { get; private set; } // Ảnh minh họa cho bước (optional)
    public Guid RecipeId { get; private set; }
}

```

```

public Recipe? Recipe { get; private set; }

protected RecipeStep() { }

public static RecipeStep Create(Guid recipeId, int stepNumber,
                                string description, int? durationMinutes =
null)
=> new() { Id=Guid.NewGuid(), RecipeId=recipeId, StepNumber=stepNumber,
           Description=description.Trim(),
           DurationMinutes=durationMinutes };
}

// — RecipeIngredient
public class RecipeIngredient
{
    public Guid Id { get; private set; }
    public string Name { get; private set; } = string.Empty; // "Thịt bò"
    public decimal Quantity { get; private set; } // 500
    public string Unit { get; private set; } = string.Empty; // "gram"
    public string? Notes { get; private set; } // "thái lát mỏng"
    (optional)
    public int SortOrder { get; private set; } // Thứ tự hiển thị
    public Guid RecipeId { get; private set; }
    public Recipe? Recipe { get; private set; }

    protected RecipeIngredient() { }

    public static RecipeIngredient Create(Guid recipeId, string name,
                                           decimal qty, string unit,
                                           string? notes = null, int sortOrder =
0)
=> new() { Id=Guid.NewGuid(), RecipeId=recipeId, Name=name.Trim(),
           Quantity=qty, Unit=unit.Trim(), Notes=notes?.Trim(),
           SortOrder=sortOrder };
}

// — RecipeNutrition (Owned Entity / Value Object)
// Owned Entity: không có Id riêng, Lưu chung bảng với Recipe
// Bất biến (immutable): tất cả properties chỉ được set qua constructor
public record RecipeNutrition(
    decimal Calories, // kcal
    decimal Protein, // gram
    decimal Carbs, // gram
    decimal Fat // gram
);

```

3.4. ENTITY FRAMEWORK CORE NÂNG CAO

EF Core cung cấp hai cách cấu hình entity: Data Annotations (attribute trực tiếp trên class/property) và Fluent API (method chaining trong `OnModelCreating` hoặc `EntityTypeConfiguration<T>`). Fluent API mạnh hơn, linh hoạt hơn và không làm "ô nhiễm"

Domain Entity với ORM-specific attributes. Trong Clean Architecture, Fluent API là lựa chọn bắt buộc.

3.4.1 IEntityTypeConfiguration — Tách biệt cấu hình

Thay vì cấu hình tất cả entities trong `OnModelCreating()` (dễ bị hàng trăm dòng code), chúng ta tách mỗi entity configuration ra một file riêng bằng cách implement `IEntityTypeConfiguration<T>`. `ApplicationDbContext` gọi `ApplyConfigurationsFromAssembly()` để tự động nạp tất cả.

```
C# CulinaryBlog.Infrastructure.Persistence.Configurations/RecipeConfiguration.cs

using CulinaryBlog.Domain.Entities;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using NpgsqlTypes;

namespace CulinaryBlog.Infrastructure.Persistence.Configurations;

public class RecipeConfiguration : IEntityTypeConfiguration<Recipe>
{
    public void Configure(EntityTypeBuilder<Recipe> builder)
    {
        builder.ToTable("Recipes");
        builder.HasKey(r => r.Id);

        // — Column constraints

        builder.Property(r => r.Title)
            .IsRequired().HasMaxLength(200);
        builder.Property(r => r.Slug)
            .IsRequired().HasMaxLength(220);
        builder.Property(r => r.Instructions)
            .HasColumnType("text"); // TEXT không giới hạn độ dài trong PostgreSQL

        // Lưu enum DifficultyLevel dưới dạng string thay vì số nguyên
        // → Dễ đọc trong DB, không bị ảnh hưởng nếu thêm/xóa giá trị enum
        builder.Property(r => r.Difficulty)
            .HasConversion<string>()
            .HasMaxLength(20);
        builder.Property(r => r.Status)
            .HasConversion<string>()
            .HasMaxLength(20);

        // — Owned Entity – RecipeNutrition

        // Lưu chung trong bảng Recipes – không có bảng riêng, không cần JOIN
        builder.OwnsOne(r => r.Nutrition, n =>
        {
            n.Property(x => x.Calories).HasColumnName("NutritionCalories")
                .HasColumnType("decimal(8,2)");
            n.Property(x => x.Protein).HasColumnName("NutritionProtein")
                .HasColumnType("decimal(6,2)");
            n.Property(x => x.Carbs).HasColumnName("NutritionCarbs")
                .HasColumnType("decimal(6,2)");
        });
    }
}
```

```

        .HasColumnType("decimal(6,2)");
n.Property(x => x.Fat).HasColumnName("NutritionFat")
  .HasColumnType("decimal(6,2)");
});

// — Full-Text Search – Computed Column


---


// PostgreSQL tự động cập nhật SearchVector khi Title hoặc Description
thay đổi
// 'simple' config: không stemming, chỉ lowercase – phù hợp tiếng Việt
builder.Property(r => r.SearchVector)
  .HasComputedColumnSql(
    "to_tsvector('simple', " +
    "unaccent(coalesce(\"Title\", '')) || ' ' || " +
    "unaccent(coalesce(\"Description\", '')))",
    stored: true); // STORED: tính toán và lưu vào disk
builder.Property(r => r.SearchVector)
  .Metadata.SetAfterSaveBehavior(PropertySaveBehavior.Ignore);

// — Concurrency Token


---


// EF Core tự động thêm WHERE RowVersion = @currentVersion khi UPDATE
// Nếu không khớp (ai đó đã sửa) → DbUpdateConcurrencyException
builder.Property(r => r.RowVersion)
  .IsRowVersion()
  .IsConcurrencyToken();

// — Relationships


---


builder.HasOne(r => r.Category)
  .WithMany(c => c.Recipes)
  .HasForeignKey(r => r.CategoryId)
  .OnDelete(DeleteBehavior.Restrict); // Không xóa Category nếu còn
Recipe

builder.HasOne(r => r.Author)
  .WithMany(u => u.Recipes)
  .HasForeignKey(r => r.AuthorId)
  .OnDelete(DeleteBehavior.SetNull); // Xóa user → giữ recipe,
AuthorId = null

builder.HasMany(r => r.Steps)
  .WithOne(s => s.Recipe)
  .HasForeignKey(s => s.RecipeId)
  .OnDelete(DeleteBehavior.Cascade); // Xóa Recipe → xóa luôn Steps

builder.HasMany(r => r.Ingredients)
  .WithOne(i => i.Recipe)
  .HasForeignKey(i => i.RecipeId)
  .OnDelete(DeleteBehavior.Cascade);

// — Indexes


---


builder.HasIndex(r => r.Slug).IsUnique();
builder.HasIndex(r => r.CategoryId);
builder.HasIndex(r => r.AuthorId);
builder.HasIndex(r => r.Status);
builder.HasIndex(r => r.CreatedAt);

```

```
// GIN index cho Full-Text Search – tra cứu  $O(\log N)$  thay vì  $O(N)$ 
builder.HasIndex(r => r.SearchVector)
        .HasMethod("GIN");
    }
}
```

3.4.2 Migration và Apply

Shell Terminal – Tạo và áp dụng Migration

```
# Tạo migration từ thư mục gốc của solution
dotnet ef migrations add InitialCreate \
    --project src/CulinaryBlog.Infrastructure \
    --startup-project src/CulinaryBlog.API \
    --output-dir Persistence/Migrations

# Kiểm tra SQL sẽ được tạo (không áp dụng vào DB)
dotnet ef migrations script \
    --project src/CulinaryBlog.Infrastructure \
    --startup-project src/CulinaryBlog.API

# Áp dụng tất cả migrations pending vào database
dotnet ef database update \
    --project src/CulinaryBlog.Infrastructure \
    --startup-project src/CulinaryBlog.API

# Cài extension unaccent cho PostgreSQL (cần thiết cho FTS tiếng Việt)
# Chạy trực tiếp trong PostgreSQL hoặc thêm vào migration
# CREATE EXTENSION IF NOT EXISTS unaccent;
```

💡 Lưu ý

Trong môi trường production, KHÔNG nên dùng dotnet ef database update trực tiếp. Thay vào đó, dùng dotnet ef migrations script để tạo file SQL, review kỹ, rồi chạy qua DBA hoặc tích hợp vào CI/CD pipeline. Chương 6 sẽ hướng dẫn tích hợp migration vào GitHub Actions.

3.5. TỐI ƯU TRUY VẤN

ORM giúp developer viết code nhanh hơn, nhưng cũng ẩn đi SQL được sinh ra. Không ít ứng dụng bị chậm vì developer không biết ORM đang thực hiện bao nhiêu SQL queries mỗi request. Phần này đi qua các vấn đề hiệu năng phổ biến nhất với EF Core và cách khắc phục.

3.5.1 N+1 Query Problem

N+1 là vấn đề hiệu năng phổ biến nhất với mọi ORM. Khi bạn truy vấn N entities và sau đó truy cập navigation property của từng entity trong vòng lặp, EF Core sẽ thực hiện 1 query cho danh sách + N queries cho từng navigation property = N+1 queries tổng cộng.

❌ Sai — N+1 Problem	✅ Đúng — Eager Loading
<pre>// TƯỜNG CHỈ 1 query, thực ra N+1 queries! var recipes = await context.Recipes .ToListAsync(); // 1 query: SELECT * FROM Recipes foreach (var r in recipes) { // Mỗi lần truy cập: thêm 1 SQL! // "SELECT * FROM Categories WHERE Id=@id" Console.WriteLine(r.Category!.Name); // Console.WriteLine(r.Author!.FullName); // Thêm 1 query nữa! } // Với 100 recipes → 201 queries!</pre>	<pre>// Chỉ 1 query duy nhất (SQL JOIN) var recipes = await context.Recipes .Include(r => r.Category) .Include(r => r.Author) .AsNoTracking() // Không cần track nếu chỉ đọc .ToListAsync(); // SQL được sinh ra: // SELECT r.*, c.*, u.* // FROM "Recipes" r // LEFT JOIN "Categories" c // ON r."CategoryId" = c."Id" // LEFT JOIN "AspNetUsers" u // ON r."AuthorId" = u."Id" // Toàn bộ data lấy trong 1 roundtrip!</pre>

Hình 3.2. N+1 Problem và cách khắc phục với Eager Loading

3.5.2 Cartesian Explosion và AsSplitQuery()

Khi Include nhiều collection navigation properties (1-N relationships), EF Core mặc định dùng JOIN — tạo ra "Cartesian product". Ví dụ: recipe có 10 Steps và 15 Ingredients sẽ tạo ra 150 rows (10×15) trong kết quả SQL, dù chỉ có 25 bản ghi thực tế. AsSplitQuery() giải quyết bằng cách chạy SQL riêng biệt cho mỗi Include.

```
C# CulinaryBlog.Infrastructure/Persistence/Repositories/RecipeRepository.cs

public async Task<Recipe?> GetDetailAsync(Guid id, CancellationToken ct =
    default)
{
    // AsSplitQuery(): chạy 4 SQL riêng biệt thay vì 1 JOIN khổng lồ
    // Query 1: SELECT recipe WHERE Id = @id + JOIN Category + JOIN Author
    // Query 2: SELECT steps WHERE RecipeId = @id ORDER BY StepNumber
    // Query 3: SELECT ingredients WHERE RecipeId = @id ORDER BY SortOrder
    // Query 4: SELECT images WHERE RecipeId = @id
    // Tổng data nhỏ hơn nhiều so với Cartesian product
    return await Db.Recipes
        .AsSplitQuery()
        .Include(r => r.Category)
        .Include(r => r.Author)
        .Include(r => r.Steps.OrderBy(s => s.StepNumber))
        .Include(r => r.Ingredients.OrderBy(i => i.SortOrder))
        .Include(r => r.Images)
        .FirstOrDefaultAsync(r => r.Id == id, ct);
}
```

3.5.3 AsNoTracking và Projection

Hai kỹ thuật tối ưu quan trọng nhất cho read-only queries: `AsNoTracking()` tắt Change Tracking (tiết kiệm bộ nhớ và CPU), còn Projection (Select hoặc `ProjectToType`) chỉ lấy đúng columns cần thiết thay vì `SELECT *`.

```
C#
CulinaryBlog.Application/Features/Recipes/Queries/GetRecipes/GetRecipesQueryHandler.cs

public async Task<PaginatedResult<RecipeListDto>> Handle(
    GetRecipesQuery request, CancellationToken ct)
{
    // Bắt đầu với IQueryable – chưa có SQL nào được thực thi
    var query = _unitOfWork.Recipes.Query()
        .AsNoTracking();           // Tắt Change Tracking: tiết kiệm ~30% bộ nhớ

    // Filter theo status – chỉ hiện Published recipes
    query = query.Where(r => r.Status == RecipeStatus.Published);

    // Filter theo category (optional)
    if (request.CategoryId.HasValue)
        query = query.Where(r => r.CategoryId == request.CategoryId);

    // Filter theo difficulty (optional)
    if (request.Difficulty.HasValue)
        query = query.Where(r => r.Difficulty == request.Difficulty);

    var totalCount = await query.CountAsync(ct); // Truy vấn COUNT riêng biệt

    // Projection: Mapper chỉ SELECT các cột có trong RecipeListDto
    // Không SELECT Instructions, SearchVector, RowVersion, ...
    // → Giảm data từ DB về app server đáng kể
    var items = await query
        .OrderByDescending(r => r.CreatedAt)
        .Skip((request.Page - 1) * request.PageSize)
        .Take(request.PageSize)
        .ProjectToType<RecipeListDto>() // Mapper Projection – compile-time
        .ToListAsync(ct);

    return new PaginatedResult<RecipeListDto>(items, totalCount,
        request.Page, request.PageSize);
}
```

3.5.4 Bật SQL Logging để quan sát query

```
SQL appsettings.Development.json (bật EF Core SQL logging)

{
  "Logging": {
    "LogLevel": {
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  }
}
```

⚡ **Hiệu năng**

Trong development, luôn bật SQL logging để quan sát query EF Core sinh ra. Nếu thấy cùng một câu SELECT lặp lại nhiều lần → đây là dấu hiệu N+1. Nếu thấy một query quá dài với nhiều JOIN → cân nhắc AsSplitQuery(). Một request API bình thường nên thực hiện không quá 5-10 SQL queries. Nếu vượt quá con số này, cần review lại.

3.6. CHỈ MỤC VÀ TỐI ƯU HÓA TÌM KIẾM

3.6.1 Các loại Index trong PostgreSQL

Index là cấu trúc dữ liệu phụ trợ giúp PostgreSQL tìm kiếm nhanh hơn — thay vì đọc tuần tự toàn bộ bảng (Sequential Scan $O(N)$), có thể tìm kiếm theo cây (B-tree $O(\log N)$) hoặc bảng hash. Đổi lại, index tốn thêm dung lượng và làm chậm INSERT/UPDATE/DELETE.

Index Type	Cấu trúc	Phù hợp với	Phù hợp trong Culinary Blog
B-tree	Balanced tree — mặc định	=, <, >, BETWEEN, LIKE xxx%	Slug (unique), CategoryId, CreatedAt
Hash	Hash table	Chỉ toán tử =, nhanh hơn B-tree	UserID (exact match) — ít dùng trong PostgreSQL
GIN	Inverted index	Array, JSONB, Full-Text Search, tsvector	SearchVector (bắt buộc dùng GIN)
GiST	Generalized tree	Geometric data, ranges, IP addresses	PostGIS extension (toạ độ địa lý)
BRIN	Block range index	Cột có giá trị tăng dần, bảng cực lớn	CreatedAt trên bảng log (hàng triệu rows)

Bảng 3.2. So sánh các loại Index trong PostgreSQL

3.6.2 Tại sao LIKE không đủ và khi nào cần Full-Text Search

Toán tử LIKE '%từ khóa%' không dùng được index B-tree (vì wildcard ở đầu), phải scan tuần tự toàn bộ bảng. Khi Culinary Blog có hàng chục nghìn công thức, query LIKE trên Title và Description sẽ cực kỳ chậm. Ngoài ra, LIKE không hiểu ngữ nghĩa: tìm "thịt bò" không tìm được "bò kho" hay "thịt kho bò".

Tiêu chí	LIKE '%keyword%'	Full-Text Search (tsvector)
Sử dụng Index	✗ Không (sequential scan)	☑ GIN Index ($O(\log N)$)
Tốc độ (100k rows)	~500ms	~5ms (với GIN index)
Tách từ (tokenize)	Không — tìm exact substring	Có — "phở bò" = ["phở", "bò"]

Tiêu chí	LIKE '%keyword%'	Full-Text Search (tsvector)
Ranking kết quả	Không có	ts_rank() — ưu tiên match nhiều lần
Xử lý dấu tiếng Việt	Không — "phở" ≠ "pho"	unaccent: "phở" tìm được "pho" (cần cấu hình)
Cú pháp nâng cao	Không	AND, OR, NOT, phrase search, prefix
Phù hợp với	Tìm kiếm đơn giản, ít data	Search engine cho ứng dụng production

Bảng 3.3. So sánh LIKE và Full-Text Search trong PostgreSQL

3.6.3 PostgreSQL Full-Text Search — tsvector và tsquery

PostgreSQL Full-Text Search hoạt động theo hai bước: (1) Tạo tsvector từ text — danh sách từ đã được normalize và đánh vị trí; (2) Tạo tsquery từ search term của user; (3) Dùng toán tử @@ để match. GIN index tăng tốc bước (3) lên hàng trăm lần.

SQL SQL – Hiểu tsvector và tsquery

```
-- Tạo tsvector: "Phở bò truyền thống" → Lexemes với vị trí
SELECT to_tsvector('simple', unaccent('Phở bò truyền thống Hà Nội'));
-- Kết quả: 'bo':2 'ha':5 'noi':6 'pho':1 'truyen':3 'thong':4

-- Tạo tsquery: chuỗi tìm kiếm của user
SELECT to_tsquery('simple', unaccent('phở & bò'));           -- AND: phải có cả hai
SELECT plainto_tsquery('simple', unaccent('phở bò'));         -- Tự parse thành AND
SELECT websearch_to_tsquery('simple', unaccent('"phở bò"'));  -- Phrase search

-- Match: recipe có chứa cả "pho" VÀ "bo" không?
SELECT "Title", ts_rank("SearchVector", plainto_tsquery('simple', 'pho bo')) AS
rank
FROM "Recipes"
WHERE "SearchVector" @@ plainto_tsquery('simple', 'pho bo')
ORDER BY rank DESC
LIMIT 10;

-- GIN index đảm bảo query trên là O(log N) thay vì O(N)
EXPLAIN ANALYZE
SELECT "Title" FROM "Recipes"
WHERE "SearchVector" @@ plainto_tsquery('simple', 'pho bo');
-- Output: Bitmap Index Scan on ix_recipes_searchvector (cost=0.00..8.02 rows=1
width=0)
-- → GIN index được sử dụng! Không có Sequential Scan
```

3.6.4 Implement Full-Text Search trong EF Core + IRecipeRepository

```
C# CulinaryBlog.Infrastructure.Persistence.Repositories/RecipeRepository.cs

using NpgsqlTypes;

public class RecipeRepository : Repository<Recipe>, IRecipeRepository
```

```

{
    public RecipeRepository(ApplicationDbContext db) : base(db) { }

    public async Task<Recipe?> GetDetailAsync(Guid id, CancellationToken ct =
default)
    {
        return await Db.Recipes
            .AsSplitQuery()
            .Include(r => r.Category)
            .Include(r => r.Author)
            .Include(r => r.Steps.OrderBy(s => s.StepNumber))
            .Include(r => r.Ingredients.OrderBy(i => i.SortOrder))
            .Include(r => r.Images)
            .FirstOrDefaultAsync(r => r.Id == id, ct);
    }

    public async Task<IReadOnlyList<Recipe>> SearchAsync(
        string searchTerm, int page = 1, int pageSize = 10, CancellationToken ct =
default)
    {
        // Normalize search term: Loại bỏ dấu, lowercase
        // EF.Functions.ToTsQuery() dịch sang SQL plainto_tsquery(...)
        var tsQuery = EF.Functions.PlainToTsQuery("simple", searchTerm);

        return await Db.Recipes
            .AsNoTracking()
            .Where(r => r.Status == RecipeStatus.Published &&
                r.SearchVector!.Matches(tsQuery)) // WHERE @@
            .OrderByDescending(r => // Sắp xếp theo
                r.SearchVector!.Rank(tsQuery)) // ts_rank()
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .Include(r => r.Category)
            .Include(r => r.Images.Take(1)) // Chỉ lấy 1 ảnh đại diện
            .ToListAsync(ct);
    }

    public async Task<IReadOnlyList<Recipe>> GetByCategoryAsync(
        Guid categoryId, int page = 1, int pageSize = 10, CancellationToken ct =
default)
    {
        return await Db.Recipes
            .AsNoTracking()
            .Where(r => r.CategoryId == categoryId &&
                r.Status == RecipeStatus.Published)
            .OrderByDescending(r => r.CreatedAt)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .Include(r => r.Author)
            .ToListAsync(ct);
    }

    public async Task<bool> SlugExistsAsync(string slug, CancellationToken ct =
default)
        => await Db.Recipes.AnyAsync(r => r.Slug == slug, ct);
    }
}

```

Database

Tiếng Việt không có stemmer riêng trong PostgreSQL. Giải pháp thực tế: (1) Dùng unaccent extension để chuẩn hóa dấu ("phở" → "pho"); (2) Dùng cấu hình "simple" thay vì "english" (không có stemming sai); (3) Để user gõ không dấu vẫn tìm được có dấu. Với yêu cầu tìm kiếm nâng cao hơn (gợi ý, fuzzy search), có thể kết hợp pg_trgm (trigram index) — nhưng đây nằm ngoài phạm vi chương này.

3.7. DATABASE SEEDING

Dữ liệu mẫu (seed data) là điều kiện tiên quyết để phát triển và kiểm thử hiệu quả. Không ai muốn viết UI và API khi database trống hoàn toàn. EF Core hỗ trợ HasData() cho dữ liệu tĩnh (categories, roles), nhưng với dữ liệu phong phú và thực tế hơn (hàng trăm recipes), thư viện Bogus là công cụ tốt nhất.

3.7.1 EF Core HasData() cho dữ liệu cố định

C# CulinaryBlog.Infrastructure/Persistence/Configurations/CategoryConfiguration.cs

```
public class CategoryConfiguration : IEntityTypeConfiguration<Category>
{
    // Danh mục cố định – không thay đổi theo môi trường
    private static readonly Guid[] CategoryIds =
    {
        Guid.Parse("10000000-0000-0000-0000-000000000001"),
        Guid.Parse("10000000-0000-0000-0000-000000000002"),
        Guid.Parse("10000000-0000-0000-0000-000000000003"),
        Guid.Parse("10000000-0000-0000-0000-000000000004"),
        Guid.Parse("10000000-0000-0000-0000-000000000005"),
    };

    public void Configure(EntityTypeBuilder<Category> builder)
    {
        builder.ToTable("Categories");
        builder.HasKey(c => c.Id);
        builder.Property(c => c.Name).IsRequired().HasMaxLength(100);
        builder.Property(c => c.Slug).IsRequired().HasMaxLength(120);
        builder.HasIndex(c => c.Slug).IsUnique();

        // HasData() – dữ liệu này được đưa vào migration và áp dụng lúc deploy
        // Dùng Guid cố định để idempotent (chạy nhiều lần vẫn cho kết quả giống nhau)
        builder.HasData(
            new { Id=CategoryIds[0], Name="Món Chính", Slug="mon-chinh",
                  Description="Các món ăn chính trong bữa cơm Việt",
                  CreatedAt=new DateTime(2025,1,1,0,0,0,DateTimeKind.Utc) },
            new { Id=CategoryIds[1], Name="Món Canh", Slug="mon-canh",
                  Description="Canh và súp truyền thống",
                  CreatedAt=new DateTime(2025,1,1,0,0,0,DateTimeKind.Utc) },
            new { Id=CategoryIds[2], Name="Món Tráng Miệng", Slug="mon-trang-
mieng",
                  Description="Các loại bánh, chè và tráng miệng",
```

```

        CreatedAt=new DateTime(2025,1,1,0,0,0,DateTimeKind.Utc) },
    new { Id=CategoryIds[3], Name="Đồ Uống",      Slug="do-uong",
        Description="Nước ép, sinh tố và thức uống",
        CreatedAt=new DateTime(2025,1,1,0,0,0,DateTimeKind.Utc) },
    new { Id=CategoryIds[4], Name="Ăn Vặt",      Slug="an-vat",
        Description="Snack và đồ ăn nhẹ",
        CreatedAt=new DateTime(2025,1,1,0,0,0,DateTimeKind.Utc) }
    );
}
}

```

3.7.2 Bogus — sinh dữ liệu thực tế và phong phú

Bogus là thư viện .NET/C# sinh dữ liệu ngẫu nhiên có cấu trúc. Khác với Random thông thường, Bogus hiểu ngữ nghĩa: `Faker.Name.FullName()` sinh tên người thật, `Faker.Internet.Email()` sinh email hợp lệ. Chúng ta sẽ dùng Bogus để sinh dữ liệu Culinary Blog mang tính thực tế cao.

```

C# CulinaryBlog.Infrastructure/Persistence/Seeders/CulinaryBlogSeeder.cs

using Bogus;
using CulinaryBlog.Domain.Entities;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;

namespace CulinaryBlog.Infrastructure.Persistence.Seeders;

public static class CulinaryBlogSeeder
{
    // Seed cố định để cùng seed luôn cho cùng dữ liệu → reproducible test
    // environment
    private const int SEED = 42;

    // Danh sách tên món ăn Việt Nam thực tế
    private static readonly string[] RecipeTitles =
        "Phở bò truyền thống Hà Nội", "Bánh mì kẹp thịt nướng", "Bún bò Huế đặc
        biệt",
        "Cơm tấm sườn bì chả", "Gỏi cuốn tôm thịt", "Bánh xèo miền Nam",
        "Canh chua cá lóc", "Thịt kho tàu", "Cà ri gà Việt Nam",
        "Chả giò chiên giòn", "Bún thịt nướng", "Bánh cuốn nhân thịt",
        "Mì Quảng tôm thịt", "Hủ tiếu Nam Vang", "Lẩu thái chua cay",
        "Bún riêu cua", "Xôi gấc đỏ", "Chè đậu đỏ nước cốt dừa",
        "Bánh flan caramel", "Trà sữa matcha trân châu",
    ];

    private static readonly string[] IngredientNames =
        "Thịt bò", "Thịt lợn", "Tôm tươi", "Cá lóc", "Trứng gà",
        "Rau muống", "Cải thảo", "Cà chua", "Hành lá", "Tỏi",
        "Gừng tươi", "Sả", "Ớt đỏ", "Nước mắm", "Dầu hào",
        "Đường", "Muối", "Tiêu đen", "Dầu ăn", "Nước dừa",
    ];

    public static async Task SeedAsync(IServiceProvider serviceProvider)
    {

```

```

        using var scope = serviceProvider.CreateScope();
        var db =
scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
        var userManager =
scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();

        // Chỉ seed nếu database chưa có dữ liệu (idempotent)
        if (await db.Recipes.AnyAsync()) return;

        // — Tạo Author users


---


        var authorFaker = new Faker<ApplicationUser>("vi")
            .RuleFor(u => u.Id, f => Guid.NewGuid().ToString())
            .RuleFor(u => u.FullName, f => f.Name.FullName())
            .RuleFor(u => u.UserName, f => f.Internet.UserName())
            .RuleFor(u => u.Email, f => f.Internet.Email())
            .RuleFor(u => u.EmailConfirmed, f => true)
            .RuleFor(u => u.AvatarUrl, f => f.Internet.Avatar());

        var authors = authorFaker.UseSeed(SEED).Generate(5);
        foreach (var author in authors)
        {
            await userManager.CreateAsync(author, "Author@123456");
            await userManager.AddToRoleAsync(author, "Author");
        }

        // — Lấy Category IDs đã seed


---


        var categoryIds = await db.Categories.Select(c => c.Id).ToListAsync();

        // — Sinh Recipes


---


        var recipeFaker = new Faker<Recipe>()
            .CustomInstantiator(f =>
            {
                // Dùng factory method thay vì new Recipe() (tuân thủ DDD)
                var title = f.PickRandom(RecipeTitles);
                var recipe = Recipe.Create(
                    title: title,
                    description: f.Lorem.Sentences(2, " "),
                    categoryId: f.PickRandom(categoryIds),
                    authorId: f.PickRandom(authors).Id,
                    prepTime: f.Random.Int(10, 45),
                    cookTime: f.Random.Int(20, 180),
                    servings: f.Random.Int(2, 8),
                    difficulty: f.PickRandom<DifficultyLevel>());

                // Publish một số recipes ngay (thực tế hơn)
                if (f.Random.Bool(0.7)) recipe.Publish();

                // Thêm thông tin dinh dưỡng cho một số recipe
                if (f.Random.Bool(0.5))
                    recipe.SetNutrition(
                        f.Random.Decimal(200, 800), // Calories
                        f.Random.Decimal(10, 60), // Protein (g)
                        f.Random.Decimal(20, 100), // Carbs (g)
                        f.Random.Decimal(5, 40)); // Fat (g)
            });

```

```

        return recipe;
    });

    var recipes = recipeFaker.UseSeed(SEED).Generate(50);

    // — Thêm Steps và Ingredients cho mỗi recipe

    var stepFaker = new Faker("vi");
    var ingrFaker = new Faker("vi");

    foreach (var recipe in recipes)
    {
        // 3-7 bước thực hiện
        var stepCount = stepFaker.Random.Int(3, 7);
        for (int s = 1; s <= stepCount; s++)
        {
            recipe.Steps.Add(RecipeStep.Create(
                recipeId: recipe.Id,
                stepNumber: s,
                description: stepFaker.Lorem.Sentences(2, " "),
                durationMinutes: stepFaker.Random.Int(2, 30)));
        }

        // 5-12 nguyên liệu
        var ingrCount = ingrFaker.Random.Int(5, 12);
        for (int i = 0; i < ingrCount; i++)
        {
            recipe.Ingredients.Add(RecipeIngredient.Create(
                recipeId: recipe.Id,
                name: ingrFaker.PickRandom(IngredientNames),
                qty: ingrFaker.Random.Decimal(50, 500),
                unit: ingrFaker.PickRandom("gram", "ml", "muỗng", "cái", "củ"),
                sortOrder: i));
        }

        await db.Recipes.AddRangeAsync(recipes);
        await db.SaveChangesAsync();

        Console.WriteLine($"Seeded: {authors.Count} authors, {recipes.Count}
recipes.");
    }
}

```

Text CulinaryBlog.API/Program.cs (gọi seeder)

```

// Gọi seeder khi khởi động trong Development
if (app.Environment.IsDevelopment())
{
    // Tự động apply pending migrations
    using var scope = app.Services.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    await db.Database.MigrateAsync();

    // Seed dữ liệu mẫu (chỉ seed nếu chưa có data)
    await CulinaryBlogSeeder.SeedAsync(app.Services);
}

```

```
}

app.Run();
```

3.8. CASE STUDY: RECIPES – INGREDIENTS – STEPS

Case Study này tổng hợp toàn bộ nội dung Chương 3: thiết lập đầy đủ tầng Infrastructure/Persistence cho Culinary Blog, từ entity configuration đến Full-Text Search đến database seeding. Sau khi hoàn thành, database sẵn sàng cho việc phát triển tính năng ở các chương sau.

Bước 1 — Đăng ký Repository và UnitOfWork vào DI

Text CulinaryBlog.Infrastructure/DependencyInjection.cs (bổ sung Repository)

```
// Trong AddInfrastructure(), thêm sau phần DbContext setup:

// Đăng ký UnitOfWork – Scoped vì DbContext là Scoped
services.AddScoped<IUnitOfWork, UnitOfWork>();

// Đăng ký Repositories riêng Lẻ (nếu có Handler inject trực tiếp)
services.AddScoped<IRecipeRepository, RecipeRepository>();
services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
```

Bước 2 — Cập nhật Handler dùng IUnitOfWork

C# CulinaryBlog.Application/Features/Recipes/Commands/CreateRecipe/CreateRecipeCommandHandler.cs

```
public class CreateRecipeCommandHandler
    : IRequestHandler<CreateRecipeCommand, RecipeDto>
{
    private readonly IUnitOfWork _unitOfWork;

    public CreateRecipeCommandHandler(IUnitOfWork unitOfWork)
        => _unitOfWork = unitOfWork;

    public async Task<RecipeDto> Handle(
        CreateRecipeCommand request, CancellationToken ct)
    {
        // Kiểm tra slug chưa tồn tại
        var slug = SlugHelper.Generate(request.Title);
        if (await _unitOfWork.Recipes.SlugExistsAsync(slug, ct))
            throw new ConflictException($"Slug '{slug}' đã tồn tại. Vui lòng dùng tiêu
            đề khác.");

        // Kiểm tra category hợp lệ
        if (!await _unitOfWork.Categories.ExistsAsync(request.CategoryId, ct))
            throw new NotFoundException(nameof(Category), request.CategoryId);

        // Tạo Recipe qua factory method
        var recipe = Recipe.Create(
```

```

        request.Title, request.Description,
        request.CategoryId, request.AuthorId,
        request.PrepTimeMinutes, request.CookTimeMinutes,
        request.Servings, request.Difficulty);

    // Thêm các bước thực hiện
    for (int i = 0; i < request.Steps.Count; i++)
    {
        var step = request.Steps[i];
        recipe.Steps.Add(RecipeStep.Create(
            recipe.Id, i + 1, step.Description, step.DurationMinutes));
    }

    // Thêm nguyên liệu
    for (int i = 0; i < request.Ingredients.Count; i++)
    {
        var ing = request.Ingredients[i];
        recipe.Ingredients.Add(RecipeIngredient.Create(
            recipe.Id, ing.Name, ing.Quantity, ing.Unit, ing.Notes, i));
    }

    await _unitOfWork.Recipes.AddAsync(recipe, ct);
    await _unitOfWork.SaveChangesAsync(ct); // Một SaveChanges duy nhất

    return recipe.Adapt<RecipeDto>();
}
}

```

Bước 3 — Full-Text Search Endpoint

Text CulinaryBlog.API/Endpoints/RecipeEndpoints.cs (Search endpoint)

```

// GET /api/v1/recipes/search?q=phở+bò&page=1&pageSize=10
group.MapGet("/search", async (
    [FromQuery] string q,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 10,
    ISender sender,
    CancellationToken ct) =>
{
    if (string.IsNullOrWhiteSpace(q))
        return Results.BadRequest("Vui lòng nhập từ khóa tìm kiếm.");

    var result = await sender.Send(
        new SearchRecipesQuery(q.Trim(), page, pageSize), ct);
    return Results.Ok(result);
})
.WithName("SearchRecipes")
.WithSummary("Tìm kiếm công thức nấu ăn bằng Full-Text Search")
.AllowAnonymous();

```

Bước 4 — Kiểm tra với EXPLAIN ANALYZE

SQL SQL – Phân tích query execution plan

```

-- Chạy trực tiếp trong pgAdmin hoặc psql để verify GIN index được dùng

```

```

EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT r."Id", r."Title", ts_rank(r."SearchVector", q) AS rank
FROM "Recipes" r,
      plainto_tsquery('simple', unaccent('pho bo')) q
WHERE r."SearchVector" @@ q
ORDER BY rank DESC
LIMIT 10;

-- Kết quả mong đợi (GIN index được dùng):
-- Limit (cost=1.58..24.65 rows=10 width=52) (actual time=0.312..0.318 rows=8
Loops=1)
--   -> Sort (cost=1.58..1.60 rows=8 width=52)
--         Sort Key: (ts_rank(r.search_vector, q.q)) DESC
--         -> Bitmap Heap Scan on "Recipes" r (cost=0.00..1.41 rows=8 width=52)
--               -> Bitmap Index Scan on "ix_recipes_searchvector"
--                     Index Cond: (search_vector @@ q.q)
-- Planning Time: 0.8 ms
-- Execution Time: 0.4 ms   ← Với index, không phải ~500ms không có index

```

CÂU HỎI THẢO LUẬN

1. Tại sao trong Clean Architecture, chúng ta cần Repository Pattern thêm vào phía trên EF Core thay vì dùng DbContext trực tiếp? Hãy giải thích từ góc độ testability và Dependency Inversion.
2. Unit of Work Pattern giải quyết vấn đề gì mà Repository Pattern một mình không giải quyết được? Cho ví dụ một use case cụ thể trong Culinary Blog cần transaction.
3. Phân tích ưu nhược điểm của IQueryable<T>.Query() trong IRepository. Có rủi ro gì khi expose IQueryable ra ngoài Infrastructure Layer không?
4. Owned Entity trong EF Core khác gì với Value Object trong DDD? Tại sao RecipeNutrition phù hợp để làm Owned Entity trong ngữ cảnh này?
5. Giải thích Cartesian Explosion xảy ra như thế nào khi Include nhiều collection. Với recipe có 10 steps và 15 ingredients, số rows trong kết quả SQL không dùng AsSplitQuery() là bao nhiêu? So sánh với khi dùng AsSplitQuery().
6. Tại sao PostgreSQL dùng cấu hình "simple" thay vì "english" cho Full-Text Search tiếng Việt? Kết quả khác nhau thế nào nếu dùng "english" để tokenize "phở bò"?
7. GIN index phù hợp với Full-Text Search nhưng tốn chi phí gì? Trong trường hợp nào không nên tạo index?
8. Bogus.UseSeed(SEED) trong CulinaryBlogSeeder có ý nghĩa gì? Nếu không dùng seed cố định, điều gì xảy ra khi chạy seeder nhiều lần?

BÀI TẬP THỰC HÀNH

Bài 1 — Repository Pattern và EF Core (Cơ bản)

- Implement đầy đủ IRepository<T>, IUnitOfWork, Repository<T> và UnitOfWork theo code mẫu trong chương.

- Viết `IEntityTypeConfiguration` cho `RecipeStep` và `RecipeIngredient` (constraints, indexes, relationships).
- Tạo và apply migration `InitialCreate`, kiểm tra schema trong `pgAdmin`.
- Viết `CreateRecipeCommandHandler` sử dụng `IUnitOfWork` (không dùng `IApplicationDbContext` nữa).
- Test bằng `Scalar`: tạo recipe với 3 steps và 5 ingredients, verify database qua `pgAdmin`.

Bài 2 — Query Optimization (Trung bình)

- Bật SQL logging trong `Development`, quan sát queries khi gọi `GET /api/v1/recipes` (có `Include` hay không?).
- Tạo endpoint `GET /api/v1/recipes/{id}` trả về `RecipeDetailDto` đầy đủ (category, author, steps, ingredients). Dùng `AsSplitQuery` và `Include` đúng cách.
- So sánh thời gian thực thi (ms) giữa query có `AsNoTracking()` và không có, với 100 recipes. Ghi lại kết quả.
- Implement `ProjectToType<RecipeListDto>()` trong `GetRecipesQueryHandler` và verify SQL chỉ `SELECT` các cột cần thiết.
- Dùng `EXPLAIN ANALYZE` trong `pgAdmin` để kiểm tra 3 queries: `GetAll`, `GetById`, `GetByCategory`. Index nào được dùng?

Bài 3 — Full-Text Search và Seeding (Nâng cao)

- Tạo migration bổ sung `SearchVector` computed column và `GIN` index (dùng đoạn SQL trong `RecipeConfiguration`).
- Implement `SearchRecipesQuery` và `SearchRecipesQueryHandler` sử dụng `EF.Functions.PlainToTsQuery()`.
- Test Full-Text Search: tìm "pho bo" (không dấu) → phải tìm thấy recipe "Phở bò" (có dấu). Giải thích tại sao `unaccent()` cần thiết.
- Mở rộng `CulinaryBlogSeeder`: thêm 10 comments cho mỗi recipe (dùng `Faker` để sinh nội dung comment thực tế).
- Viết một migration thêm full-text search cho cả trường `Instructions` (không chỉ `Title` và `Description`). Update `RecipeConfiguration` tương ứng.

TÓM TẮT NỘI DUNG CỐT LÕI

Khái niệm	Mô tả	Trong Culinary Blog
Data Mapper	Entity không chứa DB logic — <code>DbContext</code> làm trung gian	EF Core + Clean Architecture
<code>IRepository<T></code>	Abstraction cho CRUD — Application không phụ thuộc EF Core	<code>IRepository</code> , <code>IRecipeRepository</code>

Khái niệm	Mô tả	Trong Culinary Blog
IUnitOfWork	Quản lý transaction — SaveChanges một lần duy nhất	UnitOfWork inject vào Handler
Owned Entity	Value Object lưu chung bảng — không có ID riêng	RecipeNutrition trong Recipes table
Value Conversion	Lưu enum as string — dễ đọc, migration-safe	DifficultyLevel, RecipeStatus
Concurrency Token	Phát hiện concurrent edit — RowVersion	[Timestamp] byte[] RowVersion
N+1 Query Problem	N entities + N queries navigation = N+1 total	Fix: Include() + AsNoTracking()
AsSplitQuery	Tránh Cartesian Explosion khi Include nhiều collection	GetDetailAsync()
ProjectToType<T>	SELECT chỉ columns cần — giảm data transfer	GetRecipesQueryHandler
GIN Index	Inverted index cho Full-Text Search — O(log N)	ix_recipes_searchvector
tsvector/tsquery	Cấu trúc FTS của PostgreSQL + operator @@	SearchVector + EF.Functions.PlainToTsQuery
Bogus	Sinh dữ liệu thực tế có cấu trúc	CulinaryBlogSeeder — 50 recipes

Từ khóa quan trọng:

Data Mapper, Active Record, `IRepository<T>`, `IUnitOfWork`, Owned Entity, Value Conversion, `[Timestamp]` Concurrency Token, N+1 Query Problem, `Include()` Eager Loading, `AsNoTracking()`, `AsSplitQuery()`, `ProjectToType<T>()` Projection, B-tree Index, GIN Index, `tsvector`, `tsquery`, `to_tsvector()`, `plainto_tsquery()`, `@@` operator, `unaccent()`, EXPLAIN ANALYZE, `Bogus`, `UseSeed()`

TÀI LIỆU THAM KHẢO MỞ RỘNG

Tài liệu kỹ thuật chính thức:

- Microsoft Learn — EF Core Owned Entities:
<https://learn.microsoft.com/ef/core/modeling/owned-entities>
- Microsoft Learn — EF Core Value Conversions:
<https://learn.microsoft.com/ef/core/modeling/value-conversions>
- Microsoft Learn — EF Core Split Queries:
<https://learn.microsoft.com/ef/core/querying/single-split-queries>
- PostgreSQL Documentation — Full Text Search:
<https://www.postgresql.org/docs/current/textsearch.html>

- PostgreSQL Documentation — Index Types:
<https://www.postgresql.org/docs/current/indexes-types.html>

Sách và bài viết:

- Fowler, M. (2002). Patterns of Enterprise Application Architecture. — Repository, Unit of Work, Data Mapper patterns (trang 184, 322, 174).
- Vladimir Khorikov — Unit Testing Principles, Practices, and Patterns. — Lý do Repository Pattern quan trọng cho Unit Testing.
- Jimmy Bogard — No Repository Pattern Needed: <https://jimmybogard.com/life-beyond-distributed-transactions/> — Quan điểm phản biện, mở rộng tư duy.
- Bogus Documentation: <https://github.com/bchavez/Bogus> — Full API reference và examples.
- Miloslava Farkas — EF Core Performance Tips:
<https://www.milanjovanovic.tech/blog/ef-core-query-performance-improvements> — Các kỹ thuật optimize thực tế.

Hết Chương 3